

Module Interface Specification for Software Engineering

Team 11, technically functional

Matthew Huynh

Cieran Diebolt

Vaisnavi Shanthamoorthy

Maham Siddiqui

Eman Ashraf

November 12, 2025

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Symbols, Abbreviations and Acronyms

See SRS Documentation at [<https://github.com/M9Huynh/technically-functional/blob/main/docs/SRS/S—SS>]

Contents

1	Revision History	i
2	Symbols, Abbreviations and Acronyms	ii
3	Introduction	1
4	Notation	1
5	Module Decomposition	1
6	Variables	2
7	MIS	3
7.1	Module - Database Management	3
7.1.1	Uses	3
7.1.2	Syntax	4
7.1.3	Semantics	5
7.1.4	Environment Variables	6
7.1.5	Assumptions	6
7.1.6	Access Routine Semantics	6
7.1.7	Local Functions	6
7.2	Module - Generator	7
7.2.1	Uses	7
7.2.2	Syntax	7
7.2.3	Semantics	8
7.2.4	Environment Variables	8
7.2.5	Assumptions	8
7.2.6	Access Routine Semantics	8
7.3	Module - Home	9
7.3.1	Uses	9
7.3.2	Syntax	9
7.3.3	Semantics	9
7.3.4	Environment Variables	9
7.3.5	Assumptions	10
7.3.6	Access Routine Semantics	10
7.3.7	Local Functions	10
7.4	Module - Main	11
7.4.1	Uses	11
7.4.2	Syntax	11
7.4.3	Semantics	12
7.4.4	Environment variables	12
7.4.5	Assumptions	12

	7.4.6	Access Routine Semantics	12
	7.4.7	Local Functions	12
7.5		Module - Analyze	13
	7.5.1	Uses	13
	7.5.2	Syntax	13
	7.5.3	Semantics	13
	7.5.4	Environment variables	13
	7.5.5	Assumptions	14
	7.5.6	Access Routine Semantics	14
	7.5.7	Local Functions	14
7.6		Module - Feedback	14
	7.6.1	Uses	14
	7.6.2	Syntax	14
	7.6.3	Semantics	15
	7.6.4	Assumptions	15
	7.6.5	Access Routine Semantics	15
	7.6.6	Local Functions	16
	7.6.7	Local Functions	17
7.7		Module - Metrics	17
	7.7.1	Uses	17
	7.7.2	Syntax	17
	7.7.3	Semantics	17
	7.7.4	Assumptions	17
	7.7.5	Access Routine Semantics	17
	7.7.6	Local Functions	19
	7.7.7	Local Functions	19
7.8		Module - Draw	19
	7.8.1	Uses	19
	7.8.2	Syntax	19
	7.8.3	Semantics	20
	7.8.4	Assumptions	20
	7.8.5	Access Routine Semantics	20
	7.8.6	Local Functions	21
	7.8.7	Local Functions	22
7.9		Module - UI	22
	7.9.1	Uses	22
	7.9.2	Syntax	22
	7.9.3	Semantics	22
	7.9.4	Assumptions	22
	7.9.5	Access Routine Semantics	22
	7.9.6	Local Functions	24
	7.9.7	Local Functions	25

3 Introduction

The following document details the Module Interface Specifications for a Physiotherapy Companion app. The application is designed to identify users using computer vision for their prescribed physiotherapy exercise. In addition, an analysis will be performed on the user's movement and, after completion, the app will provide metrics to the user on their performance.

This tool utilizes OpenCV and MediaPipe for image recognition and then performs measurements on the number of repetitions, range of motion and perceived exertion.

Complementary documents include the [System Requirement Specifications](#) and [Module Guide](#). The full documentation and implementation can be found at <https://github.com/M9Huynh/technically-functional>.

4 Notation

The structure of the MIS for modules comes from [Hoffman and Strooper \(1995\)](#), with the addition that template modules have been adapted from [Ghezzi et al. \(2003\)](#). The mathematical notation comes from Chapter 3 of [Hoffman and Strooper \(1995\)](#). For instance, the symbol $:=$ is used for a multiple assignment statement and conditional rules follow the form $(c_1 \Rightarrow r_1 | c_2 \Rightarrow r_2 | \dots | c_n \Rightarrow r_n)$.

The following table summarizes the primitive data types used by Software Engineering.

Data Type	Notation	Description
character	char	a single symbol or digit
integer	$\mathbb{Z}$	a number without a fractional component in $(-\infty, \infty)$
natural number	$\mathbb{N}$	a number without a fractional component in $[1, \infty)$
real	$\mathbb{R}$	any number in $(-\infty, \infty)$

The specification of Software Engineering uses some derived data types: sequences, strings, and tuples. Sequences are lists filled with elements of the same data type. Strings are sequences of characters. Tuples contain a list of values, potentially of different types. In addition, Software Engineering uses functions, which are defined by the data types of their inputs and outputs. Local functions are described by giving their type signature followed by their specification.

5 Module Decomposition

The following table is taken directly from the Module Guide document for this project.

Level 1	Level 2
Hardware-Hiding	
Behaviour-Hiding	Input Parameters Output Format Output Verification Temperature ODEs Energy Equations Control Module Specification Parameters Module
Software Decision	Sequence Data Structure ODE Solver Plotting

Table 1: Module Hierarchy

6 Variables

For clarity and readability, the variables used throughout the document are provided below.

Table 2: Variable used throughout MIS

Name	Type	Description	Used in sections
name	String	Name of the user	7.1
role	String	Values can be PT or patient	7.1
email	String	Used as login ID	7.1
password	String	Set by user for login purposes	7.1
birthday	String	User's birthday for verification	7.1
license_no	Int	Verification of PT	7.1
invite_code	String	Given by PT to patient for registration	7.1
date	String	Date selected by user	7.1
time	String	Time selected by user	7.2
recent_analysis	ADT	Most recently generated analysis	7.1
recent_report	Record	Generated report of recent_video	TBD
recent_video	String (JSON)	Last video recorded by patient	TBD
video	String (JSON)	A video in the user database	TBD
report	Record	Report(s) sent to Generator module for analysis OR requested by patient	TBD

Table 2: Variable used throughout MIS (continued)

Name	Type	Description	Used in sections
ex_instruct	String	Performance instructions of requested exercise	TBD
sets	String	Number of sets given by PT	7.1
repetitions	String	Repetitions of exercise per set	7.1
rest_time	String	Rest time between sets	7.1
leg_landmarks	Array	MediaPipe output of leg	7.4
processed_frame	Image	Video frame with overlaid markers	7.4
PT_comment	String	PT's comment on patient activity	7.3

7 MIS

7.1 Module - Database Management

ADT

This module creates an instance of UserAccount_P/PT and is an adapter that connects to the user database. This database consists of these tables:

1. User data: this consists of information such as name, role, email, password, birthday, and license_no/invite.code.
2. Exercise data: this table consists of exercise instructions, maximum height (the highest point their leg can lift up to), minimum height (starting point) and UserAccount_P['Name'].
3. User activity: This will contain information about a specific patient's exercise activity. Data such as exercise duration, maximum height, number of repetitions, target area, the date the exercise was performed and its time, rest time, number of sets and analysis. Additionally, it will include a column for the patient's feedback on their exercise performance.

7.1.1 Uses

- name
- role
- email

- password
- birthday
- license_no
- invite_code
- exercise
- time
- date
- duration
- recent_video
- recent_analysis
- Sets
- repetitions
- rest_time

7.1.2 Syntax

Exported Constants

- *UserAccount_P*: ADT
- *UserAccount_PT*: ADT

Exported Access Programs

¹ *For this function, the output depends on context. For example, if this function was going to be used in another one which corresponds to the PT functionality of viewing a patient, their view would be restricted to patients under their care.*

Access Routine Name	Input	Output	Exceptions
get_userdb_info ¹	<i>Name</i>	UserAccount_P OR UserAccount_PT	User_not_found Download_error
PTaccount_delete	<i>Name</i> <i>email</i>	NA	required_field_empty patient_not_found
username_pw_match	<i>Email</i> <i>Password</i>	NA	wrong_email_ password
add_analysis.to_ user_act	<i>UserAccount_P</i> <i>Recent_analysis</i>	NA	UserNotFoundError SaveError
save_video	<i>UserAccount_P</i> <i>Recent_video</i>	NA	User_not_found Upload_error Video_too_short
get_analysis	<i>UserAccount_P</i> <i>Date</i>	Recent_analysis	User_not_found Download_error Analysis_not_found
exerc_instruct	<i>UserAccount_P</i> <i>Exercise</i>	<i>Ex_instruct</i>	User_not_found Download_error exercise_not_in_list
PT_modifies_exercise	<i>UserAccount_P</i> <i>Sets</i> <i>Repetitions</i> <i>Rest_time</i>	<i>Ex_instruct</i>	User_not_found Download_error exercise_not_in_list

Table 3: Exported Access Routines

7.1.3 Semantics

State Variables

A: an instance of a patient UserAccount

B: an instance of a patient UserAccount

C: an instance of a PT UserAccount

State Invariant

$\forall A, B, C \in UserDatabase : A[i] \neq B[j] \neq C[k]$

$0 < A < length(A) \cap A[i] \notin \emptyset$

7.1.4 Environment Variables

Screen: The application's display to the user

Touchscreen: Users will click on the screen to select their desired functionality

7.1.5 Assumptions

NA.

7.1.6 Access Routine Semantics

get_userdb_info():

NA (*There is no transition taking place*)

PTaccount_delete():

1. From the home screen, PT accesses their patient list
2. PT selects the desired patient's name
3. PT is directed to profile of patient
4. PT selects '*Delete Patient*'
5. The selected patient is deleted from the user database

7.1.7 Local Functions

Access Routine Name	Input	Output	Exceptions
account_create	<i>Name</i> <i>Role</i> <i>Birthday</i> <i>License_no/</i> <i>Invite_code</i> <i>Email</i> <i>Password</i>	UserAccount_P or UserAccount_PT	Input_format_invalid Required_field_empty Account_already_exists Incorrect_creds Invite_code_expired
set_user_info	<i>Name</i> <i>Role</i> <i>Birthday</i> <i>License_no/</i> <i>Invite_code</i> <i>Email</i> <i>Password</i>	NA	Input_format_invalid Required_field_empty Account_already_exists Incorrect_creds Invite_code_expired
send_db_analysis	<i>UserAccount_P</i>	Recent_analysis	User_not_found Download_error
send_db_video	<i>UserAccount_P</i>	Recent_video	User_not_found Download_error
set_analysis_report	<i>UserAccount_P</i>	Recent_analysis	User_not_found Download_error

Table 4: Account Management Access Routines

7.2 Module - Generator

This module's purpose is to display the most recent report, an overall analysis of progress, and rehabilitation insights.

7.2.1 Uses

- Recent_report
- Report_db
- UserAccount_P

7.2.2 Syntax

Exported Constants

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
generate_insights	NA	NA	<i>No_activity_history</i> <i>Cannot_connect_API</i>
generate_report	<i>Date</i> <i>Time</i>	<i>Recent_report</i>	NA
get_report	<i>Date</i> <i>Time</i>	<i>Report</i>	NA

Table 5: Generator Access Routines

7.2.3 Semantics

State Variables

A : an instance of UserAccount_P

E : an instance of Recent_analysis

V : video

R : report

RR : recent_report

State Invariant

$\forall RR, \exists(A \cap V)$

$\forall R, \exists(A \cap V_i | i \geq 1..n, n \in 1..\mathbb{N})$

7.2.4 Environment Variables

NA

7.2.5 Assumptions

7.2.6 Access Routine Semantics

$[\text{accessProg} \text{---SS}]()$:

- transition: $[\text{if appropriate} \text{---SS}]$
- output: $[\text{if appropriate} \text{---SS}]$
- exception: $[\text{if appropriate} \text{---SS}]$

7.3 Module - Home

This module displays past exercises, allows viewing of user profile, and directs to the Generator module. PTs are able to comment on their patient's submissions.

7.3.1 Uses

7.3.2 Syntax

Exported Constants

NA

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
view_profile	<i>Name</i>	UserAccount_P OR UserAccount_PT	User_not_found Download_error
display_report	<i>Name</i> <i>email</i>	NA	required_field_empty patient_not_found
display_summary	<i>Email</i> <i>Password</i>	NA	wrong_email_ password
p_past_exercises	<i>Email</i>	NA	no_activity
PT_feedback_on_P	<i>PTcomment</i>	NA	User_not_found Download_error

Table 6: Home Exported Access Routines

7.3.3 Semantics

State Variables

A: an instance of UserAccount

B: an instance of UserAccount

State Invariant

NA

7.3.4 Environment Variables

NA

7.3.5 Assumptions

7.3.6 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

7.3.7 Local Functions

Access Routine Name	Input	Output	Exceptions
get_profile_gen	NA	UserAccount_P OR UserAccount_PT	Cannot_access_db

Table 7: Account Management Access Routines

7.4 Module - Main

This module handles the video recording, placing markers on it and checking if the patient's form is appropriate for performing the exercise. It does this through the use of external libraries (notably OpenCV).

7.4.1 Uses

7.4.2 Syntax

Exported Constants

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: Gets metrics from metrics module			
get_metrics	<i>NA</i>	<i>angle</i> <i>height</i> <i>duration</i>	DataRetrievalError
Description: Sends overall analysis to home module			
overall_analysis_home	<i>name</i> <i>birthday</i>	<i>overall_analysis</i>	DataRetrievalError AnalysisNotFoundError
Description: Sends real time processed frames to metrics (to be forwarded to Analysis)			
analysis_rt_frame	<i>processed_frame</i>	<i>instructions_ui</i>	CannotAnalyzeError NoInputError
Description: Performs overall analysis (given at end of recording)			
analysis_overall	<i>recent_video</i>	recent_analysis	InsufficientDataError ProcessError
Description: Module will provide corrections (received from Metrics) to UI (e.g. angle is too small)			
corrections_metrics	<i>rt_metrics</i>	<i>NA</i>	User_not_found Download_error
Description: Sends recent analysis to home module			
send_analysis_home	<i>NA</i>	recent_analysis	UserNotFoundError

Table 8: Access Routines for Main Module

Access Routine Name	Input	Output	Exceptions
Description: Calls draw to give markers of the leg			
set_markers	unprocessed_frame	processed_frame	InvalidFrameError

Table 9: Access Routines for Main Module (Continued)

7.4.3 Semantics

State Variables

State Invariant

7.4.4 Environment variables

camera

7.4.5 Assumptions

7.4.6 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

7.4.7 Local Functions

Access Routine Name	Input	Output	Exceptions
Description: Gets starting form of user			
get_initial_form	<i>input_frame</i>	unprocessed_frame	InvalidFrameError

Table 10: Local Access Routines (Main)

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

7.5 Module - Analyze

7.5.1 Uses

7.5.2 Syntax

Exported Constants

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: gets metrics from metrics module			
latest_metrics	<i>rt_metrics</i>	<i>NA</i>	InsufficientDataError
Description: sends overall analysis to Feedback module			
send_analysis_to_feedback	<i>recent_analysis</i>	<i>NA</i>	AnalysisNotFoundError
Description: performs real time analysis			
analysis_rt	<i>processed_frames</i>	recent_analysis	User_not_found Download_error
Description: overall analysis (given at end of recording)			
analysis_overall	<i>video</i>	recent_analysis	ConnectionError InsufficientDataError
Description: module will correct form (eg. if angle is too small)			
adjust_metrics	<i>processed_frame</i>	rt_metrics	UndetectedFormError
Description: send overall analysis to Database module			
send_analysis_to_db	<i>overall_analysis</i>	<i>NA</i>	User_not_found Download_error

Table 11: Access Routines for Analysis Module

7.5.3 Semantics

State Variables

TBD

State Invariant

TBD

7.5.4 Environment variables

NA

7.5.5 Assumptions

7.5.6 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

7.5.7 Local Functions

Access Routine Name	Input	Output	Exceptions
Description: Gets initial landmarks from the starting position			
get_initial_landmarks	NA	UserAccount_P OR UserAccount_PT	User_not_found Download_error

Table 12: Access Routines for Analysis Module

7.6 Module - Feedback

NA

7.6.1 Uses

7.6.2 Syntax

Exported Constants

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
ex_feedback_to_main	NA	UserAccount_P OR UserAccount_PT	User_not_found Download_error

7.6.3 Semantics

State Variables

State Invariant

7.6.4 Assumptions

7.6.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

7.6.6 Local Functions

Access Routine Name	Input	Output	Exceptions
placeholder	<i>Name</i> <i>Role</i> <i>Birthday</i> <i>License_no/</i> <i>Invite_code</i> <i>Email</i> <i>Password</i>	UserAccount_P or UserAccount_PT	Input_format_invalid Required_field_empty Account_already_exists Incorrect_creds Invite_code_expired

Table 14: Account Management Access Routines

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

7.6.7 Local Functions

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

7.7 Module - Metrics

NA

7.7.1 Uses

7.7.2 Syntax

Exported Constants

Exported Access Programs

Access Name	Routine	Input	Output	Exceptions
placeholder		NA	UserAccount_P OR UserAccount_PT	User_not_found Download_error

7.7.3 Semantics

State Variables

State Invariant

7.7.4 Assumptions

7.7.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]

- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

7.7.6 Local Functions

Access Routine Name	Input	Output	Exceptions
placeholder	<i>Name</i> <i>Role</i> <i>Birthday</i> <i>License_no/</i> <i>Invite_code</i> <i>Email</i> <i>Password</i>	UserAccount_P or UserAccount_PT	Input_format_invalid Required_field_empty Account_already_exists Incorrect_creds Invite_code_expired

Table 16: Account Management Access Routines

7.7.7 Local Functions

7.8 Module - Draw

NA

7.8.1 Uses

7.8.2 Syntax

Exported Constants

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
record_check	NA	UserAccount_P OR UserAccount_PT	User_not_found Download_error
form_check	NA	UserAccount_P OR UserAccount_PT	User_not_found Download_error
draw_points	NA	UserAccount_P OR UserAccount_PT	User_not_found Download_error

7.8.3 Semantics

State Variables

State Invariant

7.8.4 Assumptions

7.8.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

7.8.6 Local Functions

Access Routine Name	Input	Output	Exceptions
mpipe_draw	<i>Name</i> <i>Role</i> <i>Birthday</i> <i>License_no/</i> <i>Invite_code</i> <i>Email</i> <i>Password</i>	UserAccount_P or UserAccount_PT	Input_format_invalid Required_field_empty Account_already_exists Incorrect_creds Invite_code_expired

Table 18: Account Management Access Routines

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

7.8.7 Local Functions

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

7.9 Module - UI

NA

7.9.1 Uses

7.9.2 Syntax

Exported Constants

Exported Access Programs

Access Name	Routine	Input	Output	Exceptions
placeholder		NA	UserAccount_P OR UserAccount_PT	User_not_found Download_error

7.9.3 Semantics

State Variables

State Invariant

7.9.4 Assumptions

7.9.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]

- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

7.9.6 Local Functions

Access Routine Name	Input	Output	Exceptions
placeholder	<i>Name</i> <i>Role</i> <i>Birthday</i> <i>License_no/</i> <i>Invite_code</i> <i>Email</i> <i>Password</i>	UserAccount_P or UserAccount_PT	Input_format_invalid Required_field_empty Account_already_exists Incorrect_creds Invite_code_expired

Table 20: Account Management Access Routines

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

7.9.7 Local Functions

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

References

Carlo Ghezzi, Mehdi Jazayeri, and Dino Mandrioli. *Fundamentals of Software Engineering*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2003.

Daniel M. Hoffman and Paul A. Strooper. *Software Design, Automated Testing, and Maintenance: A Practical Approach*. International Thomson Computer Press, New York, NY, USA, 1995. URL <http://citeseer.ist.psu.edu/428727.html>.

8 Appendix

[Extra information if required —SS]

Appendix — Reflection

[Not required for CAS 741 projects —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?
4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), if any, needed to be changed, and why?
5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO_ProbSolutions)
6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design? (LO_Explores)

Group - Reflection

1. N/A
2. N/A
3. N/A

4. I think that our other documents such as the [requirements](#), [hazard analysis](#) need to be slightly adjusted as a result of the design documents. With the requirements specifically some of them pertain to how the application design is and after completing our initial draft of the MG and MIS, the design does not necessarily line up with the requirements document. Further, some of the hazards considered should be changed to include a couple more areas for hazards to occur. Lastly for both of the documents, the assumptions should be changed to incorporated design changes that we had not considered during that deliverable.
5. TODO
6. TODO

Matthew - Reflection

1. I think something that went well was our changes to overall workflow. One thing that we recognized was starting our document on google docs and then transferring our to sections to GitHub on the last two days the the deliverable was due led to some things going unchecked and the deliverable not turning out the way we had planned. Further it had also resulted in a 'panic' when we had to resolve merge conflicts between our sections soon to the deadline. Overall, we cut google docs for this deliverable and mainly focused on our sections within GitHub and doing multiple small PRs within the deliverable timeline and that had gone quite well. There are a few days left but a majority of the document has at least been accounted for and hopefully this workflow works for our team.
2. I think one pain point I experienced was time management. I had suggested that since the computing team's workload was quite different to the documentation team's, we could take on small sections within the document with hopes of submitting it early into the project. This did partially go well but with focus split in two areas it was hard to focus on the development side of things for the POC demo. This may be something that improves with time but as of now is currently a work in progress as development on my end is not where I want it to be.
3. I think that our design decisions partially came from speaking with our stakeholder (Kevin Yu) but other decisions were made with typical application creation. For example, the high-level was talked through with Kevin to simulate the timing and feedback delivery that you would encounter during physiotherapy but things like reporting, logging-in as well as account creation were from looking at other applications and aspects that went well from that end.
4. Group Q
5. Group Q

6. Group Q